

DAY 12 — CTEs (Common Table Expressions)

Today we'll learn one of the most useful SQL techniques for writing **clean, maintainable, production-quality queries**:

WITH ...

A CTE lets you break a complicated SQL query into **named steps**.

You've already seen how subqueries can become deeply nested. Today we'll turn those nested queries into something much easier to read.

1. What Is a CTE?

CTE means:

Common Table Expression

Basic syntax:

```
WITH cte_name AS (  
    SELECT ...  
)  
SELECT ...  
FROM cte_name;
```

Example:

```
WITH user_documents AS (  
    SELECT  
        user_id,  
  
        COUNT(*) AS document_count  
    FROM documents  
    GROUP BY user_id  
)  
SELECT *  
FROM user_documents;
```

Think of it as:

Step 1

↓

Create temporary named result

↓

Step 2

↓

Use that result

2. Why Do We Need CTEs?

Compare this complicated subquery:

```
SELECT
user_id,

document_count

FROM (
  SELECT
    user_id,

    COUNT(*) AS document_count
  FROM documents
  GROUP BY user_id
) user_totals
WHERE document_count > (
  SELECT AVG(document_count)
  FROM (
    SELECT
      user_id,

      COUNT(*) AS document_count
    FROM documents
    GROUP BY user_id
  ) totals
);
```

It works, but it's difficult to read.

With a CTE:

```
WITH user_totals AS (
  SELECT
    user_id,

    COUNT(*) AS document_count
  FROM documents
  GROUP BY user_id
)
SELECT
```

```

user_id,

document_count

FROM user_totals
WHERE document_count > (
    SELECT AVG(document_count)
    FROM user_totals
);

```

Much easier.

3. CTE Mental Model

Think of a CTE as giving a query a name.

```
WITH user_totals AS (...)
```

creates a named result:

user_totals

```

Then:
SELECT *
FROM user_totals;

```

uses it.

Conceptually:

documents

↓

GROUP BY user

↓

user_totals

↓

outer query

4. Basic CTE Example

Suppose we want total pages per user.

Without CTE:

```

SELECT
user_id,

SUM(page_count) AS total_pages

```

```

FROM documents
GROUP BY user_id;
With CTE:
WITH user_totals AS (
  SELECT
    user_id,

    SUM(page_count) AS total_pages
  FROM documents
  GROUP BY user_id
)
SELECT *
FROM user_totals;

```

The result is the same.

The advantage is that now user_totals can be used as a logical building block.

5. CTE + WHERE

```

WITH user_totals AS (
  SELECT
    user_id,

    SUM(page_count) AS total_pages
  FROM documents
  GROUP BY user_id
)
SELECT
  user_id,

  total_pages

FROM user_totals
WHERE total_pages > 100;

```

Logical flow:

documents

↓

SUM pages by user

↓

user_totals

↓

WHERE total_pages > 100

6. CTE + ORDER BY

```
WITH user_totals AS (  
  SELECT  
    user_id,  
  
    SUM(page_count) AS total_pages  
  FROM documents  
  GROUP BY user_id  
)  
SELECT  
user_id,  
  
total_pages  
  
FROM user_totals  
ORDER BY total_pages DESC;
```

7. CTE + LIMIT

```
WITH user_totals AS (  
  SELECT  
    user_id,  
  
    SUM(page_count) AS total_pages  
  FROM documents  
  GROUP BY user_id  
)  
SELECT  
user_id,  
  
total_pages  
  
FROM user_totals  
ORDER BY total_pages DESC  
LIMIT 5;
```

This gives:

Top 5 users by total document pages.

8. CTE With JOIN

CTEs become especially useful when combining multiple tables.

```
WITH document_stats AS (  

```

```

SELECT
  user_id,

  COUNT(*) AS document_count,
  SUM(page_count) AS total_pages,
  AVG(page_count) AS average_pages
FROM documents
GROUP BY user_id
)
SELECT
  u.id,

  u.name,

  ds.document_count,

  ds.total_pages,

  ds.average_pages

FROM users u
JOIN document_stats ds
  ON u.id = ds.user_id;

```

Now the query has two logical stages:

documents

↓

document_stats

↓

JOIN users

↓

final result

9. Why This Is Better Engineering

Imagine a production query containing:

10 JOINS

5 aggregations

4 subqueries

3 CASE statements

A giant query can become difficult to:

- debug
- review
- modify
- test
- explain
- maintain

CTEs allow you to write:

Step 1 → document statistics

Step 2 → chunk statistics

Step 3 → embedding statistics

Step 4 → combine

Step 5 → classify

This maps naturally to Data Engineering pipelines.

10. Multiple CTEs

You can define multiple CTEs.

Syntax:

```
WITH
cte_one AS (

    ...

),
cte_two AS (

    ...

),
cte_three AS (

    ...

)
SELECT ...
FROM cte_one
JOIN cte_two
```

```
ON ...  
JOIN cte_three  
ON ...;
```

Example:

```
WITH document_stats AS (  
  SELECT  
    user_id,  
  
    COUNT(*) AS document_count,  
    SUM(page_count) AS total_pages  
  FROM documents  
  GROUP BY user_id  
)  
user_info AS (  
  SELECT  
    id,  
  
    name,  
  
    country  
  FROM users  
)  
SELECT  
ui.id,  
  
ui.name,  
  
ui.country,  
  
ds.document_count,  
  
ds.total_pages  
  
FROM user_info ui  
JOIN document_stats ds  
  ON ui.id = ds.user_id;
```

11. CTEs Can Depend on Previous CTEs

This is extremely important.

Example:

```
WITH user_totals AS (  
  SELECT  
    user_id,
```

```

        SUM(page_count) AS total_pages
    FROM documents
    GROUP BY user_id
),
large_users AS (
    SELECT
        user_id,
        total_pages
    FROM user_totals
    WHERE total_pages > 100
)
SELECT *
FROM large_users;

```

Relationship:

documents

↓

user_totals

↓

large_users

↓

final SELECT

The second CTE can use the first.

12. A More Realistic Example

Suppose we want:

Users with more than 2 documents and more than 100 total pages.

We can create:

```

WITH user_stats AS (
    SELECT
        user_id,
        COUNT(*) AS document_count,
        SUM(page_count) AS total_pages
    FROM documents
    GROUP BY user_id
)

```

```

)
SELECT
user_id,

document_count,

total_pages

FROM user_stats
WHERE document_count > 2
AND total_pages > 100;

```

This is much easier to understand than repeating the aggregation.

13. CTE vs Subquery

Subquery:

```

SELECT *
FROM (
    SELECT
        user_id,

        SUM(page_count) AS total_pages
    FROM documents
    GROUP BY user_id
) user_totals
WHERE total_pages > 100;

```

CTE:

```

WITH user_totals AS (
    SELECT
        user_id,

        SUM(page_count) AS total_pages
    FROM documents
    GROUP BY user_id
)
SELECT *
FROM user_totals
WHERE total_pages > 100;

```

Both are valid.

The CTE version gives the intermediate result a meaningful name.

14. CTE Does NOT Automatically Mean "Permanent Table"

This is important.

When you write:

WITH user_totals AS (...)

you are **not creating a permanent database table**.

It exists for the duration of that SQL statement.

Think:

Permanent table



stored in database

CTE



temporary query-level result

Don't confuse a CTE with:

CREATE TABLE

15. CTE Does NOT Automatically Mean Faster

Very important:

CTEs are primarily a **query organization/readability technique**.

Don't assume:

CTE = faster query

or:

subquery = slower query

PostgreSQL's optimizer can transform many query structures.

Performance depends on:

- query plan
- data size
- indexes

- joins
- filtering
- aggregation
- PostgreSQL version
- CTE materialization behavior

We'll study this deeply in **Day 20 — EXPLAIN and Query Optimization**.

16. CTE + RAG Pipeline

Now let's model a real RAG pipeline.

Tables:

users

documents

document_chunks

embeddings

We want:

For each document, calculate chunk count and embedding count.

First CTE:

```
WITH chunk_stats AS (
  SELECT
    document_id,

    COUNT(*) AS chunk_count
  FROM document_chunks
  GROUP BY document_id
)
SELECT
  d.id,

  d.title,

  cs.chunk_count

FROM documents d
LEFT JOIN chunk_stats cs
  ON d.id = cs.document_id;
```

This is already much cleaner.

17. Multiple CTEs for RAG

Now let's calculate:

chunk_count

embedding_count

ready_embeddings

failed_embeddings

We can build separate logical stages.

```
WITH chunk_stats AS (  
  SELECT  
    document_id,  
  
    COUNT(*) AS chunk_count  
  FROM document_chunks  
  GROUP BY document_id  
)  
embedding_stats AS (  
  SELECT  
    c.document_id,  
  
    COUNT(e.id) AS embedding_count,  
    COUNT(  
      CASE  
        WHEN e.vector_status = 'ready'  
        THEN e.id  
      END  
    ) AS ready_embeddings,  
    COUNT(  
      CASE  
        WHEN e.vector_status = 'failed'  
        THEN e.id  
      END  
    ) AS failed_embeddings  
  FROM document_chunks c  
  LEFT JOIN embeddings e  
    ON c.id = e.chunk_id  
  GROUP BY c.document_id  
)  
SELECT  
  d.id,
```

```

d.title,

cs.chunk_count,

es.embedding_count,

es.ready_embeddings,

es.failed_embeddings

FROM documents d
LEFT JOIN chunk_stats cs
    ON d.id = cs.document_id
LEFT JOIN embedding_stats es
    ON d.id = es.document_id;

```

This query has a clear structure:

document_chunks

↓

chunk_stats

document_chunks

↓

embeddings

↓

embedding_stats

documents

↓

combine statistics

18. Handling NULL With COALESCE

Because we're using LEFT JOIN, a document with no chunks might produce:

```
chunk_count = NULL
```

But for reporting, you might want:

```
chunk_count = 0
```

Use:

COALESCE(cs.chunk_count, 0)

Example:

```
WITH chunk_stats AS (  
  SELECT  
    document_id,  
  
    COUNT(*) AS chunk_count  
  FROM document_chunks  
  GROUP BY document_id  
)  
SELECT  
  d.id,  
  
  d.title,  
  
  COALESCE(cs.chunk_count, 0) AS chunk_count  
  
FROM documents d  
LEFT JOIN chunk_stats cs  
  ON d.id = cs.document_id;
```

Remember:**NULL**

↓

COALESCE

↓

0

19. COALESCE With Multiple Values

Syntax:

COALESCE(value1, value2, value3)

It returns the first non-NULL value.

Example:

COALESCE(cs.chunk_count, 0)

means:

If chunk_count exists, use it. Otherwise use 0.

This is extremely common in analytics queries.

20. CTE + CASE

Now classify pipeline status.

```
WITH chunk_stats AS (  
  SELECT  
    document_id,  
  
    COUNT(*) AS chunk_count  
  FROM document_chunks  
  GROUP BY document_id  
)  
SELECT  
  d.id,  
  
  d.title,  
  
  COALESCE(cs.chunk_count, 0) AS chunk_count,  
  
  CASE  
    WHEN COALESCE(cs.chunk_count, 0) = 0  
    THEN 'NOT_CHUNKED'  
    ELSE 'CHUNKED'  
  END AS chunking_status  
FROM documents d  
LEFT JOIN chunk_stats cs  
  ON d.id = cs.document_id;
```

This is a very realistic pipeline dashboard query.

21. CTE + Multiple JOINS

Suppose:

users

documents

chunks

embeddings

Instead of joining everything immediately, you can aggregate each child table first.

```

WITH chunk_stats AS (
  SELECT
    document_id,

    COUNT(*) AS chunk_count
  FROM document_chunks
  GROUP BY document_id
),
embedding_stats AS (
  SELECT
    c.document_id,

    COUNT(e.id) AS embedding_count
  FROM document_chunks c
  LEFT JOIN embeddings e
    ON c.id = e.chunk_id
  GROUP BY c.document_id
)
SELECT
  d.id,

  d.title,

  u.name,

  COALESCE(cs.chunk_count, 0) AS chunk_count,

  COALESCE(es.embedding_count, 0) AS embedding_count

FROM documents d
JOIN users u
  ON d.user_id = u.id
LEFT JOIN chunk_stats cs
  ON d.id = cs.document_id
LEFT JOIN embedding_stats es
  ON d.id = es.document_id;

```

This technique helps avoid some of the row-multiplication problems you encountered on Day 10.

22. Why Aggregate Before Joining?

Consider:

Document A

↓

100 chunks

↓

100 embeddings

If you directly join:

documents

JOIN chunks

JOIN embeddings

you need to carefully reason about row multiplication.

Instead:

chunks

↓

aggregate per document

↓

chunk_stats

embeddings

↓

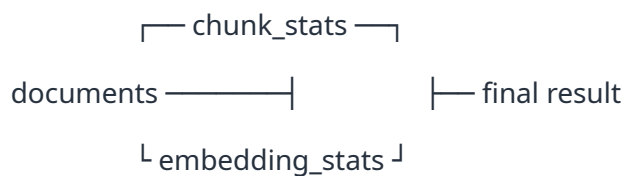
aggregate per document

↓

embedding_stats

then JOIN the smaller results

Conceptually:



This is an important advanced SQL design pattern.

23. CTE for "Above Average" Analysis

Yesterday we solved:

Users whose document count is greater than average.

Today we can make it much cleaner.

```
WITH user_counts AS (  
    SELECT  
        user_id,  
  
        COUNT(*) AS document_count  
    FROM documents  
    GROUP BY user_id  
)  
average_count AS (  
    SELECT  
        AVG(document_count) AS avg_documents  
    FROM user_counts  
)  
SELECT  
    uc.user_id,  
  
    uc.document_count  
FROM user_counts uc  
CROSS JOIN average_count ac  
  
WHERE uc.document_count > ac.avg_documents;
```

Let's understand it.

Step 1

documents

↓

user_counts

Step 2

user_counts

↓

average_count

Step 3

user_counts

+

average_count

↓

comparison

24. Why CROSS JOIN Here?

average_count contains one row:

avg_documents

2.4

We want that one value available to every row in user_counts.

So:

CROSS JOIN average_count

makes the average available to each user.

If:

user_counts

has:

5 rows

and:

average_count

has:

1 row

the result has:

$5 \times 1 = 5$ rows

This is a controlled CROSS JOIN.

25. CTEs Are Great for Debugging

One of the biggest advantages of CTEs:

You can test each step separately.

Suppose:

```
WITH user_stats AS (  
  ...  
),  
large_users AS (  
  ...  
),  
final_data AS (  
  ...  
)  
SELECT *  
FROM final_data;
```

If the output is wrong, temporarily run:

```
SELECT *  
FROM user_stats;  
Then:  
SELECT *  
FROM large_users;  
Then:  
SELECT *  
FROM final_data;
```

You can inspect the pipeline step by step.

This is very useful in production debugging.

26. CTE Naming

Good:

user_stats

document_stats

chunk_stats

embedding_stats

active_users

processed_documents

Bad:

x

temp1

abc

query2

data

Names should describe the result.

Think like a software engineer:

A good name reduces the amount of explanation needed.

27. CTE Scope

A CTE exists only within the query where it is defined.

Example:

```
WITH user_stats AS (  
  ...  
)  
SELECT *  
FROM user_stats;
```

After this query finishes:

user_stats

doesn't exist as a permanent object.

This won't work later:

```
SELECT *  
FROM user_stats;
```

unless you define the CTE again.

28. CTE Column Names

You can explicitly name CTE columns:

```
WITH user_stats(user_id, document_count) AS (  
  SELECT  
    user_id,
```

```
        COUNT(*)
    FROM documents
    GROUP BY user_id
)
SELECT *
FROM user_stats;
```

Usually it's clearer to alias columns inside the query:

```
WITH user_stats AS (
    SELECT
        user_id,

        COUNT(*) AS document_count
    FROM documents
    GROUP BY user_id
)
SELECT *
FROM user_stats;
```

29. CTE + WHERE + GROUP BY + HAVING

Example:

For processed documents, find users with at least 2 documents and more than 50 total pages.

```
WITH processed_user_stats AS (
    SELECT
        user_id,

        COUNT(*) AS document_count,
        SUM(page_count) AS total_pages
    FROM documents
    WHERE status = 'processed'
    GROUP BY user_id
    HAVING COUNT(*) >= 2
)
SELECT
    user_id,

    document_count,

    total_pages
FROM processed_user_stats
WHERE total_pages > 50;
```

Notice the logic:

documents

↓

WHERE processed

↓

GROUP BY user

↓

HAVING >= 2

↓

processed_user_stats

↓

WHERE total_pages > 50

This is exactly the kind of query decomposition you should start practicing.

30. CTE vs Temporary Table

Don't confuse:

WITH ...

with:

CREATE TEMP TABLE ...

CTE

query-scoped

Temporary table

session-scoped

A temporary table physically represents a database object for the session.

A CTE is a query construct.

We'll stay focused on CTEs today.

31. Advanced Preview: Recursive CTE

PostgreSQL also supports:

WITH RECURSIVE

This can be used for:

- organizational hierarchies
- tree structures
- graph traversal
- folder structures
- dependency chains

Example:

```
WITH RECURSIVE numbers AS (  
  SELECT 1 AS n  
  
  UNION ALL  
  
  SELECT n + 1  
  FROM numbers  
  WHERE n < 5  
)  
SELECT *  
FROM numbers;
```

Result:

```
1  
2  
3  
4  
5
```

Do not focus on recursive CTEs yet.

Your current goal is mastering normal CTEs.

32. Common CTE Mistakes

Mistake 1 — Forgetting the final SELECT

This is incomplete:

```
WITH user_stats AS (  
  SELECT ...  
)
```

A CTE normally needs a statement using it:

```
WITH user_stats AS (  
    SELECT ...  
)  
SELECT *  
FROM user_stats;
```

Mistake 2 — Referencing a CTE before defining it

Bad:

```
WITH second AS (  
    SELECT *  
    FROM first  
)  
first AS (  
    SELECT ...  
)  
SELECT *  
FROM second;
```

For normal CTE dependencies, define them in the logical order required.

Mistake 3 — Treating a CTE as a permanent table

This won't work after the statement finishes:

```
SELECT *  
FROM user_stats;
```

unless user_stats is an actual table/view or you define it again.

Mistake 4 — Recreating row multiplication

CTEs don't automatically solve bad JOIN logic.

This can still be wrong:

```
WITH everything AS (  
    SELECT *  
    FROM documents d  
    JOIN document_chunks c  
        ON d.id = c.document_id  
    JOIN embeddings e  
        ON c.id = e.chunk_id  
)
```

```
SELECT COUNT(d.id)
```

```
FROM everything;
```

You still need to understand the row structure.

33. Debugging Challenge #1

A developer writes:

```
WITH user_stats AS (  
  SELECT  
    user_id,  
  
    COUNT(*) AS document_count  
  FROM documents  
  GROUP BY user_id  
)
```

```
WITH large_users AS (  
  SELECT *  
  FROM user_stats  
  WHERE document_count > 2  
)
```

```
SELECT *  
FROM large_users;
```

What's wrong?

Hint:

How do multiple CTEs normally appear in one statement?

34. Debugging Challenge #2

Developer wants:

Documents with more than 50 pages.

They write:

```
WITH large_documents AS (  
  SELECT  
    id,  
  
    title,
```

```
page_count
FROM documents
)
SELECT *
FROM large_documents
WHERE page_count > 50;
```

Is this wrong?

Technically, no.

But ask:

Does this CTE provide meaningful value?

Explain whether you would keep or remove it.

This is about **good SQL engineering**, not just syntax.

35. Debugging Challenge #3

Developer wants document and chunk statistics.

They write:

```
WITH stats AS (
  SELECT
    d.id,

    COUNT(c.id) AS chunk_count,
    COUNT(e.id) AS embedding_count
  FROM documents d
  LEFT JOIN document_chunks c
    ON d.id = c.document_id
  LEFT JOIN embeddings e
    ON c.id = e.chunk_id
  GROUP BY d.id
)
SELECT *
FROM stats;
```

Could:

chunk_count

be incorrect?

Why?

Think about the row multiplication problem from Day 10.

36. SQL Exercises

Use:

users

documents

document_chunks

embeddings

Q1

Create a CTE named:

user_document_stats

containing:

user_id

document_count

total_pages

average_pages

Then return the CTE.

Q2

Using a CTE, find users with:

document_count >= 2

Q3

Using a CTE, find users with:

total_pages > 100

Q4

Create:

document_stats

containing:

document_id

chunk_count

Then join it to documents.

Documents with no chunks must remain.

Q5

Use COALESCE so documents with no chunks show:

chunk_count = 0

instead of NULL.

Q6

Create a CTE containing:

user_id

document_count

Then return users with document counts greater than 1.

37. Intermediate Exercises

Q7

Create:

user_stats

containing:

user_id

document_count

total_pages

average_pages

Then join it to users and return:

user_id

name

document_count

total_pages

average_pages

Q8

Create a CTE for processed documents.

Then find users who have:

at least 2 processed documents

and:

more than 50 processed pages

Q9

Create:

chunk_stats

with:

document_id

chunk_count

and:

embedding_stats

with:

document_id

embedding_count

ready_embeddings

failed_embeddings

Then combine them with documents.

Q10

Return every document with:

document_id

document_title

chunk_count

embedding_count

pipeline_status

Define:

chunk_count = 0

→ **NOT_CHUNKED**

chunk_count > 0

AND embedding_count = 0

→ **NOT_EMBEDDED**

failed_embeddings > 0

→ **EMBEDDING_ERROR**

otherwise

→ **READY**

Use CTEs + CASE.

Q11

Using a CTE, find the top 5 users by:

total_pages

Return:

user_id

user_name

document_count

total_pages

Q12

Using a CTE, find users whose:

document_count

is greater than the **average document count across users who have documents.**

38. Engineering Challenge — RAG Pipeline Dashboard

Build a production-style query.

You need one row per document:

document_id

document_title

owner_name

document_status

chunk_count

embedding_count

ready_embeddings

failed_embeddings

pipeline_status

Use these stages:

CTE 1

Create:

chunk_stats

with:

document_id

chunk_count

CTE 2

Create:

embedding_stats

with:

document_id

embedding_count

ready_embeddings

failed_embeddings

Final query

Join:

documents

users

chunk_stats

embedding_stats

Use:

COALESCE

CASE

LEFT JOIN

Your pipeline status should be:

NOT_CHUNKED

NOT_EMBEDDED

EMBEDDING_ERROR

READY

Important

Try to avoid this structure:

documents

↓

chunks

↓

embeddings

directly in the final aggregation.

Instead:

chunks

↓

chunk_stats

chunks + embeddings

↓

embedding_stats

documents

+

users

+

chunk_stats

+

embedding_stats

↓

final dashboard

This is your first serious **CTE-based data engineering query**.

39. Engineering Challenge 2 — Above Average

Create a CTE:

user_stats

containing:

user_id

document_count

total_pages

Then calculate:

average_document_count

from user_stats.

Finally return users whose document count is above the average.

Output:

user_id

user_name

document_count

average_document_count

You should solve this using **multiple CTEs**.

40. Interview Questions

Answer in your own words.

Q13

What is a CTE?

Q14

Why might a CTE be preferable to a deeply nested subquery?

Q15

Does using a CTE automatically make a query faster?

Q16

What is the scope of a normal CTE?

Q17

Can one CTE reference another CTE?

Q18

Why is COALESCE commonly used with CTEs and LEFT JOINs?

Q19

How can CTEs help with debugging?

Q20

Why can aggregating child tables in separate CTEs help avoid row multiplication?

41. Explanation Challenge

Explain this query line by line:

```
WITH chunk_stats AS (  
  SELECT  
    document_id,  
  
    COUNT(*) AS chunk_count  
  FROM document_chunks  
  GROUP BY document_id  
)  
embedding_stats AS (  
  SELECT  
    c.document_id,  
  
    COUNT(e.id) AS embedding_count,  
    COUNT(  
      CASE  
        WHEN e.vector_status = 'ready'  
        THEN e.id  
      END  
    ) AS ready_embeddings,  
    COUNT(  
      CASE  
        WHEN e.vector_status = 'failed'  
        THEN e.id  
      END  
    ) AS failed_embeddings  
  FROM document_chunks c  
  LEFT JOIN embeddings e  
    ON c.id = e.chunk_id  
  GROUP BY c.document_id  
)  
SELECT  
  d.id,  
  
  d.title,  
  
  COALESCE(cs.chunk_count, 0) AS chunk_count,  
  
  COALESCE(es.embedding_count, 0) AS embedding_count,  
  
  COALESCE(es.ready_embeddings, 0) AS ready_embeddings,  
  
  COALESCE(es.failed_embeddings, 0) AS failed_embeddings  
FROM documents d  
LEFT JOIN chunk_stats cs
```

```
ON d.id = cs.document_id
LEFT JOIN embedding_stats es
ON d.id = es.document_id;
```

Your explanation must cover:

1. Why chunk_stats exists.
2. Why embedding_stats exists.
3. Why aggregation happens inside the CTEs.
4. Why LEFT JOIN is used inside embedding_stats.
5. Why COALESCE is needed.
6. Why the final query starts from documents.
7. Why documents without chunks remain.
8. Why this structure helps with row multiplication.

42. Data Engineering Thinking

Imagine:

documents = 500 million

chunks = 5 billion

embeddings = 5 billion

Compare these two approaches.

Approach A

documents

↓

JOIN chunks

↓

JOIN embeddings

↓

GROUP BY documents

Approach B

chunks

↓

aggregate

↓

chunk_stats

chunks + embeddings

↓

aggregate

↓

embedding_stats

documents

+

chunk_stats

+

embedding_stats

Think about:

1. Which approach produces large intermediate results?
2. Why can aggregation before joining be useful?
3. Does Approach B automatically guarantee better performance?
4. What would you inspect in PostgreSQL to verify performance?

The answer to #4 is something we'll learn in **Day 20: EXPLAIN**.

43. Short Test

Q21

What does CTE stand for?

- A. Common Table Expression
- B. Combined Table Engine
- C. Common Temporary Entity
- D. Column Table Expression

Q22

Which syntax starts a CTE?

A.

CREATE CTE

B.

WITH

C.

CTE

D.

TEMP

Q23

Does a normal CTE create a permanent database table?

A. Yes

B. No

Explain.

Q24

Can this work?

```
WITH a AS (  
    SELECT ...  
),  
b AS (  
    SELECT *  
    FROM a  
)  
SELECT *  
FROM b;
```

A. Yes

B. No

Explain.

Q25

Why might this be useful?

`COALESCE(chunk_count, 0)`

Q26

What is one major reason for aggregating `document_chunks` and `embeddings` separately before joining their statistics?

- A. To make SQL shorter
- B. To avoid/reduce row multiplication problems
- C. To remove indexes
- D. To eliminate GROUP BY